

1. Lernzeit

Inhaltsverzeichnis

1. Lernzeit	1
2. Grundlegende Problemstellung	3
3. Architekturvorschlag	3
4. Technologieauswahl	5
5. Use Cases	6
6. Muss-/Kann-Kriterien	6
7. Umsetzung / Implementierung	7
7.1. Frontend (React)	7
7.1.1. Designprozess (Figma)	7
7.1.2. Komponenten	9
7.1.3. API-Kommunikation und State-Management	9
7.2. Backend (.NET API)	10
7.2.1. Architekturprinzipien	10
7.2.2. API-Endpunkte	12
7.2.3. Kalenderdaten-Verarbeitung	13
7.2.4. Gruppenverwaltung	13
7.2.5. Datenbank	13
7.2.5.1. Datenbankstruktur	13
7.2.5.2. Implementierung	14
7.2.6. Authentifizierung (Google Auth)	14
7.2.7. Kommunikation mit der RaumZeit API	14
7.3. Deployment und Infrastruktur	14
7.3.1. Deployment-Umgebung	14
7.3.2. Containerisierung und Orchestrierung mit .NET Aspire	15
8. Fazit	15
8.1. Rückblick	15
8.2. Ausblick	16
9. Literaturverzeichnis	16
Bibliografie	16

2. Grundlegende Problemstellung

Studierende stehen regelmäßig vor der Herausforderung, gemeinsame Termine für Lern- oder Projektgruppen zu koordinieren. Aufgrund individueller und häufig stark unterschiedlicher Stundenpläne gestaltet sich die Abstimmung geeigneter Zeitfenster als zeitaufwendig und ineffizient.

In der Praxis erfolgt die Terminfindung meist über Gruppenchats oder persönliche Absprachen. Dabei werden Vorschläge von einzelnen Gruppenmitgliedern eingebracht und anschließend von den übrigen Teilnehmenden auf mögliche Konflikte geprüft. Dieser iterative Abstimmungsprozess führt häufig zu einem „Hin und Her“-Austausch, der sowohl zeitintensiv als auch organisatorisch aufwendig ist.

Die Anwendung **Lernzeit** adressiert dieses Problem, indem sie die Terminfindung automatisiert unterstützt. Nutzerinnen und Nutzer können Gruppen erstellen und ihre individuell blockierten Zeiten hinterlegen. Auf Basis dieser Informationen generiert die Anwendung geeignete Terminvorschläge, zu denen alle Gruppenmitglieder verfügbar sind. Ziel ist es, den Abstimmungsaufwand zu reduzieren und eine effiziente sowie transparente Terminplanung zu ermöglichen.

3. Architekturvorschlag

Es wurden im Rahmen des Pflichtenhefts verschiedene Systemarchitekturen verglichen. Eine Gegenüberstellung hierzu ist in Tabelle 1 zu finden.

Es wurde sich im Rahmen des Projekts für eine Client-Server Architektur entschieden. Die erhöhte Komplexität alternativer Architekturstile steht in keinem angemessenen Verhältnis zu deren potenziellen Vorteilen für das Lernzeitprojekt im Rahmen des Labors.

Durch eine konsequente modulare Trennung der Funktionalitäten wird jedoch sichergestellt, dass eine spätere Migration zu einer Microservice-Architektur grundsätzlich möglich ist.

Architektur	Pro	Kontra
Client-Server	• Zentrale Kontrolle und Verwaltung • Einfache Implementierung • Gute Sicherheit durch Backend • Anonymisierung zentral möglich	• Single Point of Failure • Skalierungsprobleme bei hoher Last • Kann ggf. zu unwartbarem Monolithen wachsen
Microservice Architektur	• Gute Skalierbarkeit • Klare Trennung der Verantwortlichkeiten • Services unabhängig entwickelbar • Flexibel erweiterbar	• Höhere Komplexität • Aufwendiges Deployment • Kommunikation zwischen Services notwendig • Fehler schwerer zu debuggen
Peer-to-Peer	• Keine zentrale Instanz nötig • Gute Skalierbarkeit • Geringe Serverlast • Hohe Ausfallsicherheit	• Komplexe Client-Logik • Sicherheits- und Datenschutzprobleme • Synchronisation schwierig • Abhängigkeit von Client-Verfügbarkeit

Tabelle 1: Vergleich System Architekturen

Als Protokoll zur Kommunikation wird eine REST-Schnittstelle verwendet. Es wurde als Alternative GraphQL betrachtet, allerdings scheint auch hier durch die höhere Komplexität bei der Implementierung im Rahmen des Projekts keinen Mehrwert zu bieten.

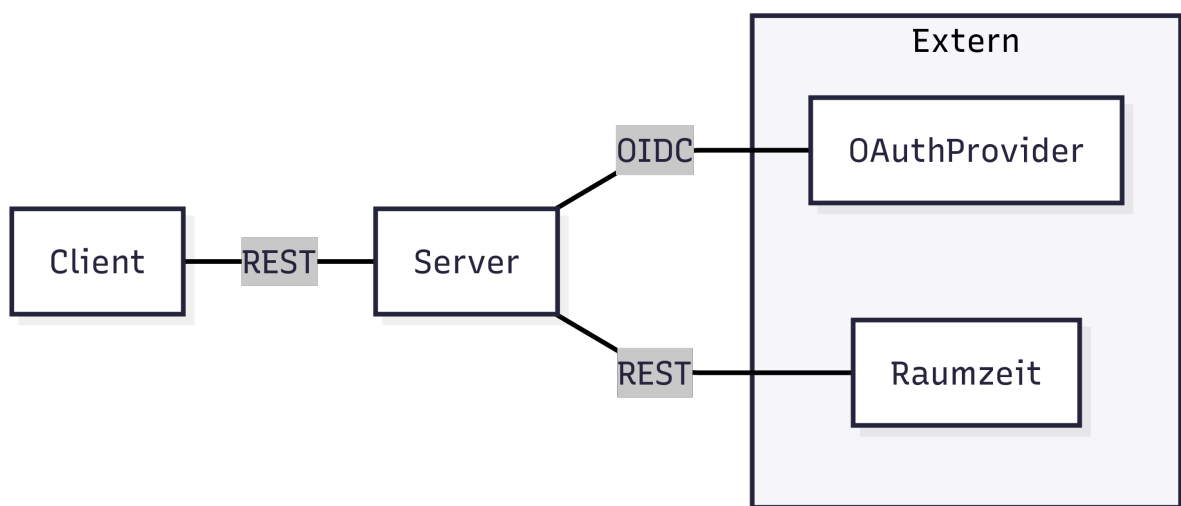


Abbildung 1: Systemarchitektur Lernzeit

4. Technologieauswahl

Im Rahmen der Analyse wurden für das Backend die Programmiersprachen *Rust*, *Go* und *C#* sowie für das Frontend die Frameworks *Blazor*, *Vue* und *React* evaluiert. Die Tabellen Tabelle 2 und Tabelle 3 listen die jeweiligen Stärken und Schwächen übersichtlich auf.

	Pro	Kontra
Rust	• Höchste Performance • Memory-Safe (Ownership & Borrowing)	• Noch keine Erfahrung im Team • Steile Lernkurve • Saubere Schichtentrennung komplex • Wenige SDKs / externe Bibliotheken
Go	• Erfahrung im Team • Sehr schnelle Entwicklung & einfache Syntax • Einfaches Deployment / kleine Container • Minimalistisch	• Fördert keine saubere Architektur (wenig Guidance) • DI nur durch externe Bibliothek • Weniger Features für komplexe Patterns
C#	• Erfahrung im Team • Ausgereiftes Ecosystem, viele SDKs • Built-in DI & klare Layer-Struktur	• Container größer, Startup langsamer • Framework kann „Magic“ verstecken

Tabelle 2: Vergleich Backend Technologien

	Pro	Kontra
Blazor	• Fullstack mit C# Backend möglich • Starke Typisierung & Compile-Time Checks • Gute Integration in .NET Ecosystem	• Wenig Erfahrung im Team • Weniger reifes Frontend-Ökosystem als JS Frameworks • Lernkurve für WebAssembly-spezifische Konzepte • Weniger Community-Beispiele & Tutorials
Vue	• Erfahrung im Team • Sehr einfache Lernkurve, leicht verständlich • Flexible Komponentenstruktur • Große Community & viele Plugins	• Architektur nicht erzwungen • Bei großen Projekten kann State-Management komplex werden • TypeScript optional, sonst lose Typisierung
React	• Erfahrung im Team • Riesiges Ecosystem & Community • Flexibles Komponentenmodell & Hooks • TypeScript-Integration möglich	• Kein festes Architektur-Muster vorgegeben • Boilerplate bei komplexem State / Redux • Lernkurve bei Hooks & Patterns für Anfänger

Tabelle 3: Vergleich Frontend Technologien

Nach Abwägung der Vor- und Nachteile fiel die Wahl auf die Kombination C# / ASP.NET als Backend-Framework und React als Frontend-Framework.

Diese Entscheidung basiert auf folgenden Kriterien:

- **Architektur:** C# ermöglicht klare Layer-Strukturen und eingebaute Dependency Injection, während React eine flexible Komponentenstruktur bietet, die eine saubere Trennung von Zuständen und Logik unterstützt.
- **Team-Expertise:** Beide Technologien sind im Team bekannt, was eine schnelle Entwicklung und Wissenstransfer erleichtert.
- **Ökosystem & Community:** React und C# bieten jeweils ein umfangreiches Ökosystem, was die Implementierung von komplexen Anforderungen erleichtert. Beide Technologien sind etabliert und haben große Communities.
- **Lern- und Weiterentwicklung:** Die Kombination erlaubt es dem Team, vorhandenes Wissen zu vertiefen und neue Best Practices im Bereich Softwarearchitektur gemeinsam zu erlernen.

5. Use Cases

1. Stundenplan importieren: Die App kann den eigenen Uni-Stundenplan übernehmen. Dafür meldet man sich auf der Hochschul-Plattform an, erstellt einen Freigabe-Link und fügt diesen in Lernzeit ein.

2. Lerngruppe erstellen: Um gemeinsame Termine zu planen, erstellt man eine neue Lerngruppe und gibt ihr einen aussagekräftigen Namen - zum Beispiel „Mathe-Lerngruppe“.

3. Teilnehmer einladen: Wer eine Gruppe erstellt hat, kann andere über einen Link oder QR-Code weitere Personen einladen. Der QR-Code dient zu einer schnellen und unkomplizierten Art einer Lerngruppe beizutreten.

4. Freie Zeiten finden und buchen: Die App zeigt Zeiten, in denen alle Gruppenmitglieder frei sind. Diese freien Zeiten kann man als gemeinsame Lernzeit auswählen und optional einen Treffpunkt sowie eine Uhrzeit festlegen.

6. Muss-/Kann-Kriterien

Mindestanforderungen	• Importieren des eigenen Stundenplanes • Erstellen von Lerngruppe • Hinzufügen von neuen Mitgliedern durch einen Einladungslink oder einen QR-Code • Abgleich von allen Kalenderdaten der Gruppenmitglieder • Visuell überschaubare Darstellung der freien Zeitblöcke
Nice to have	• Möglichkeit zur Abstimmung eines Termins • Anzeige aller frei verfügbarer Räume auf dem Campus • Benachrichtigungen bei neuen Terminvorschlägen oder Änderungen • Integration von Kalender-Apps (Google Calendar, Outlook)

7. Umsetzung / Implementierung

7.1. Frontend (React)

7.1.1. Designprozess (Figma)

Zu Beginn wurde ein Prototyp in Figma erstellt. Dabei wurde das in Figma integrierte KI-Tool genutzt, um die Vorteile von KI im Designprozess zu nutzen. Dies ermöglichte eine schnellere Erledigung von Routineaufgaben (Resizing, Layouts, Styles), ein schnelleres Testen mehrerer Ideen und Designvarianten sowie Unterstützung bei barrierefreiem und inklusivem Design. [1]

Dabei gab es jedoch auch Herausforderungen: Das resultierende Design wirkte weniger durchdacht und generisch.



Abbildung 2: Iniziale Seite von Figma AI

Mithilfe des KI-Tools wurde eine grobe Oberflächenstruktur erarbeitet, die anschließend verfeinert wurde. Dazu wurde der User-Flow von Seite zu Seite skizziert und sich am KI-generierten Seitenaufbau orientiert. (Bild von Skizzen)

Um das visuelle Erscheinungsbild der Oberfläche festzulegen, wurde ein Moodboard erstellt.

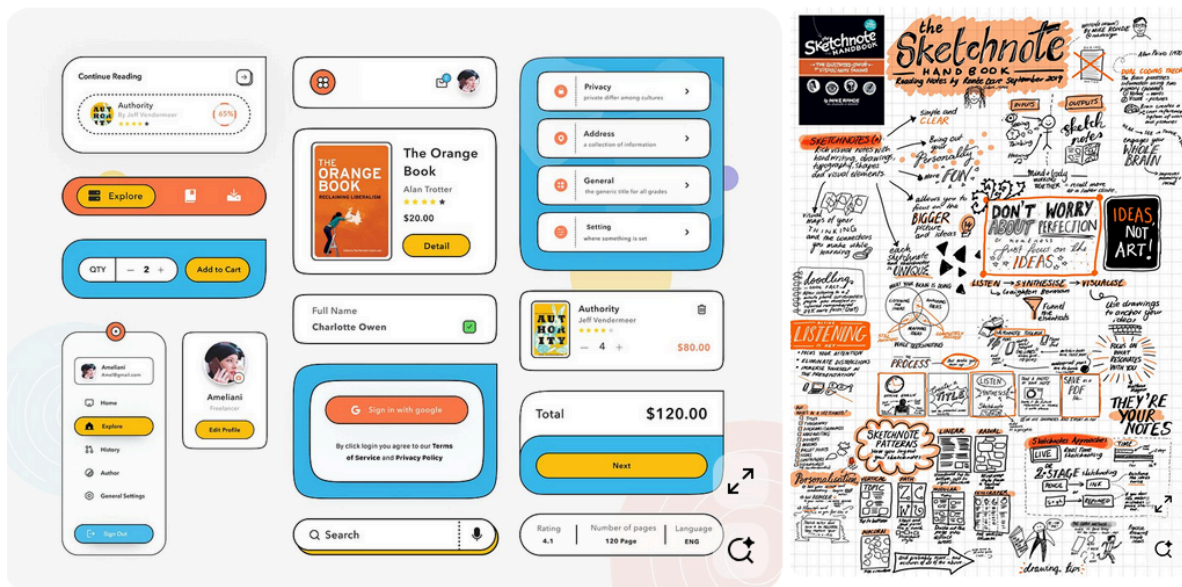


Abbildung 3: „Iniziale Seite von Figma AI“, [2], [3]

Die Referenzen helfen dabei, die visuellen Komponenten im Designprozess greifbar zu machen. Von den Referenzen ausgehend, wurden die einzelnen Komponenten erstellt und in den Seiten zusammengeführt.

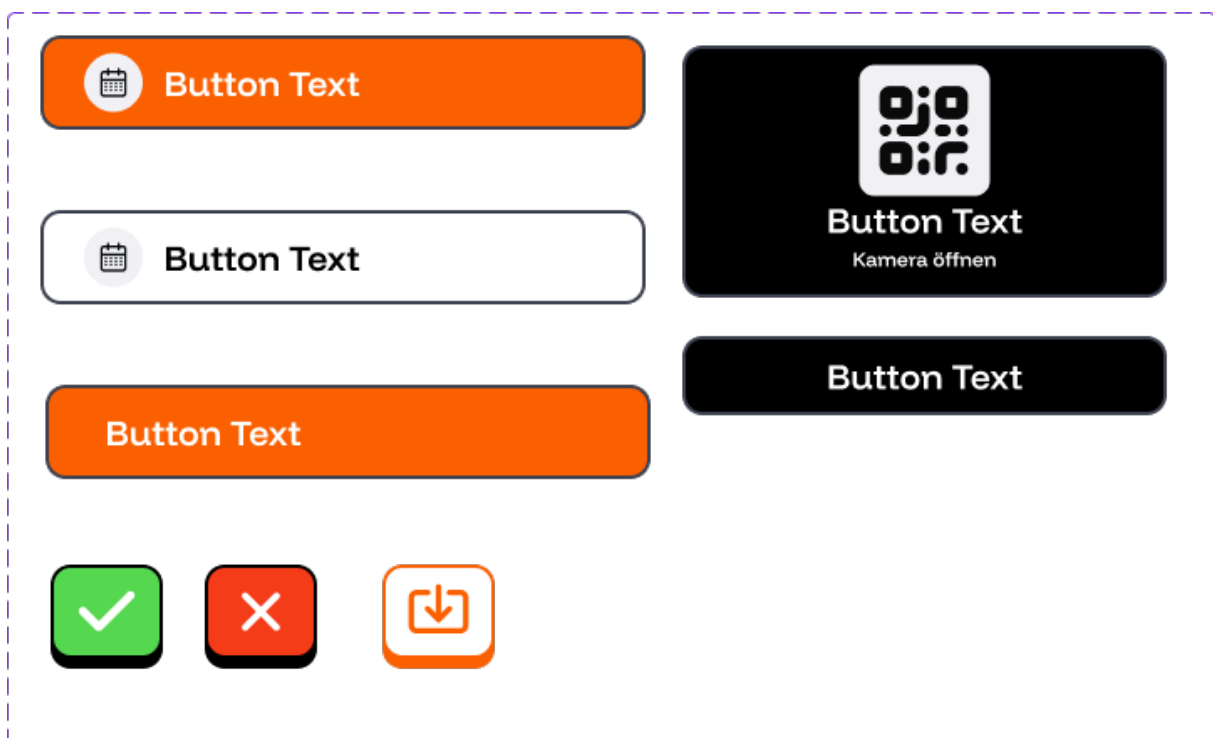


Abbildung 4: Iniziale Seite von Figma AI

Die Seitenstruktur ist wie folgt aufgebaut: (UserFlow-Diagramm einfügen)

7.1.2. Komponenten

Die Frontend-Architektur von **Lernzeit** folgt einem streng modularen Ansatz unter Verwendung von React. Das Ziel war es, eine hochgradig interaktive Benutzeroberfläche zu schaffen, die komplexe Kalenderdaten verständlich visualisiert.

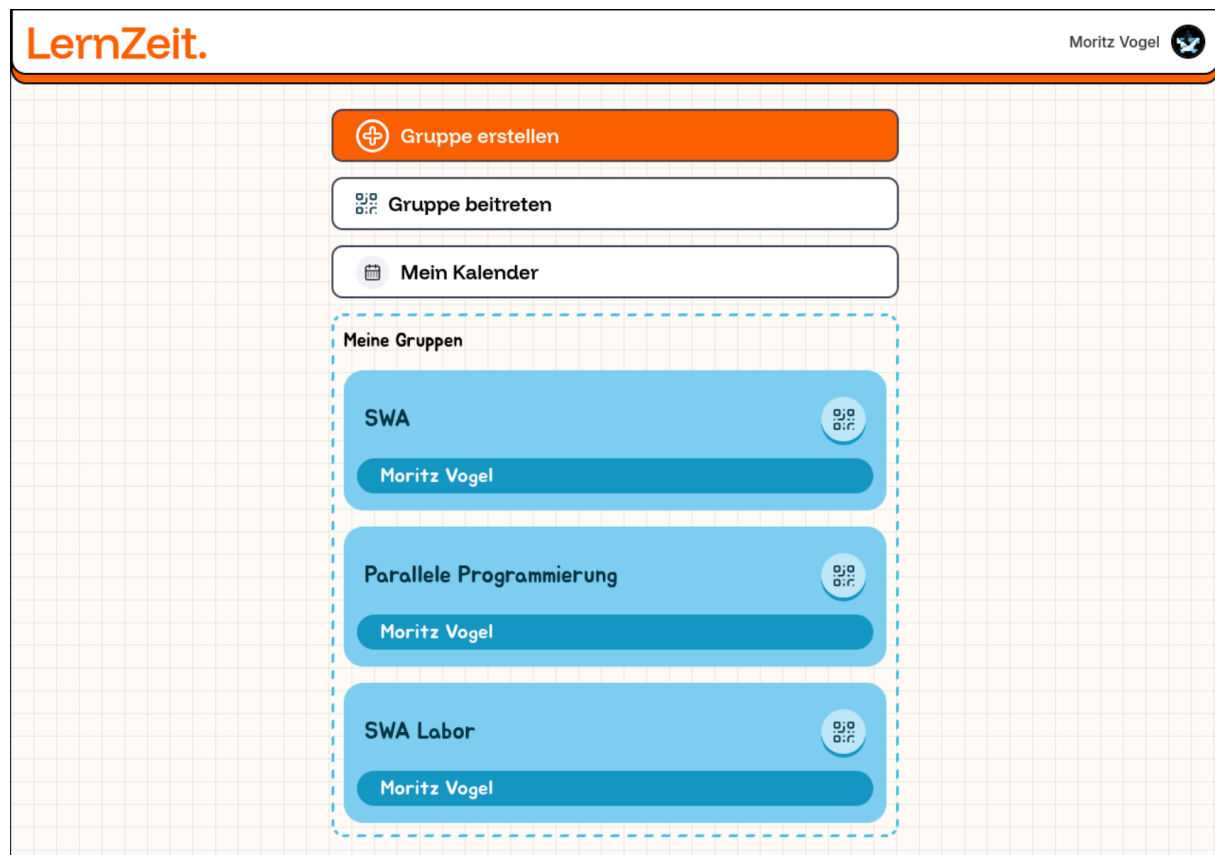


Abbildung 5: Übersicht der Hauptkomponenten in der Web-Oberfläche

Zentrale Bausteine der UI sind:

- **Timetable-Komponente:** Dies ist das Herzstück der Anwendung. Sie visualisiert Zeitfenster in einem wöchentlichen Raster. Die Logik zur Berechnung der relativen Positionen der Event-Boxen ist in der Komponente gekapselt.
- **GroupCard:** Ein wiederverwendbares Element zur Darstellung von Gruppeninformationen, das den schnellen Wechsel zwischen verschiedenen Lerngruppen ermöglicht.
- **Interaktive Modals:** Für die Erstellung von Gruppen und die Einladung von Mitgliedern wurden spezialisierte Pop-Up-Komponenten entwickelt, die einen geführten Workflow bieten.
- **Header & Navigation:** Eine konsistente Navigationsleiste, die den Nutzerstatus (Login/Logout via Google) reflektiert und schnellen Zugriff auf die Profil- und Gruppeneinstellungen bietet.

7.1.3. API-Kommunikation und State-Management

Die Anbindung an das Backend erfolgt über eine dedizierte Schicht im Frontend, die in `client.ts` definiert ist. Wir setzen hierbei auf die native `fetch-API`, ergänzt um `Error-Handling-Wrapper`.

TODO: Sequenzdiagramm eines API-Requests (z.B. GetGroupCalendar) hier einfügen

Abbildung: Fluss eines Datenabrufs vom UI-Trigger bis zum Backend-Response

Wichtige Aspekte der Implementierung sind:

- **Asynchrone Hooks:** Daten werden mittels React-Hooks (useState, useEffect) geladen. Für komplexere Datenflüsse wurden eigene Hooks wie useTimetableICS erstellt, die die Transformation von Rohdaten in das für die UI benötigte Format übernehmen.
- **Typisierung:** Durch den Einsatz von TypeScript werden die DTOs (Data Transfer Objects) des Backends im Frontend gespiegelt, was die Fehlerquote bei der Datenverarbeitung massiv reduziert.
- **Fehlerbehandlung:** Jeder Request prüft den HTTP-Statuscode. Bei Fehlern (z.B. 401 Unauthorized oder 500 Server Error) erhält der Nutzer über die UI direktes Feedback.

7.2. Backend (.NET API)

Das Backend ist als ASP.NET Core Web API realisiert und folgt den Prinzipien der Clean Architecture (Trennung von Domain, Application und Infrastruktur).

7.2.1. Architekturprinzipien

Strukturell haben wir uns im Backend für eine hexagonale Architektur (auch bekannt als „Ports und Adapter“) entschieden. Hierbei werden spezifische Technologien in Adaptern abgekapselt und somit strikt von der Domänenlogik separiert. Die verschiedenen Schichten und Adapter werden in C# als separate Projekte modelliert. Hierdurch können Zugriffe der Schichten in falscher Richtung, also von innen nach außen, strukturell verhindert werden. Im folgenden Abschnitt werden die verschiedenen Schichten erläutert:

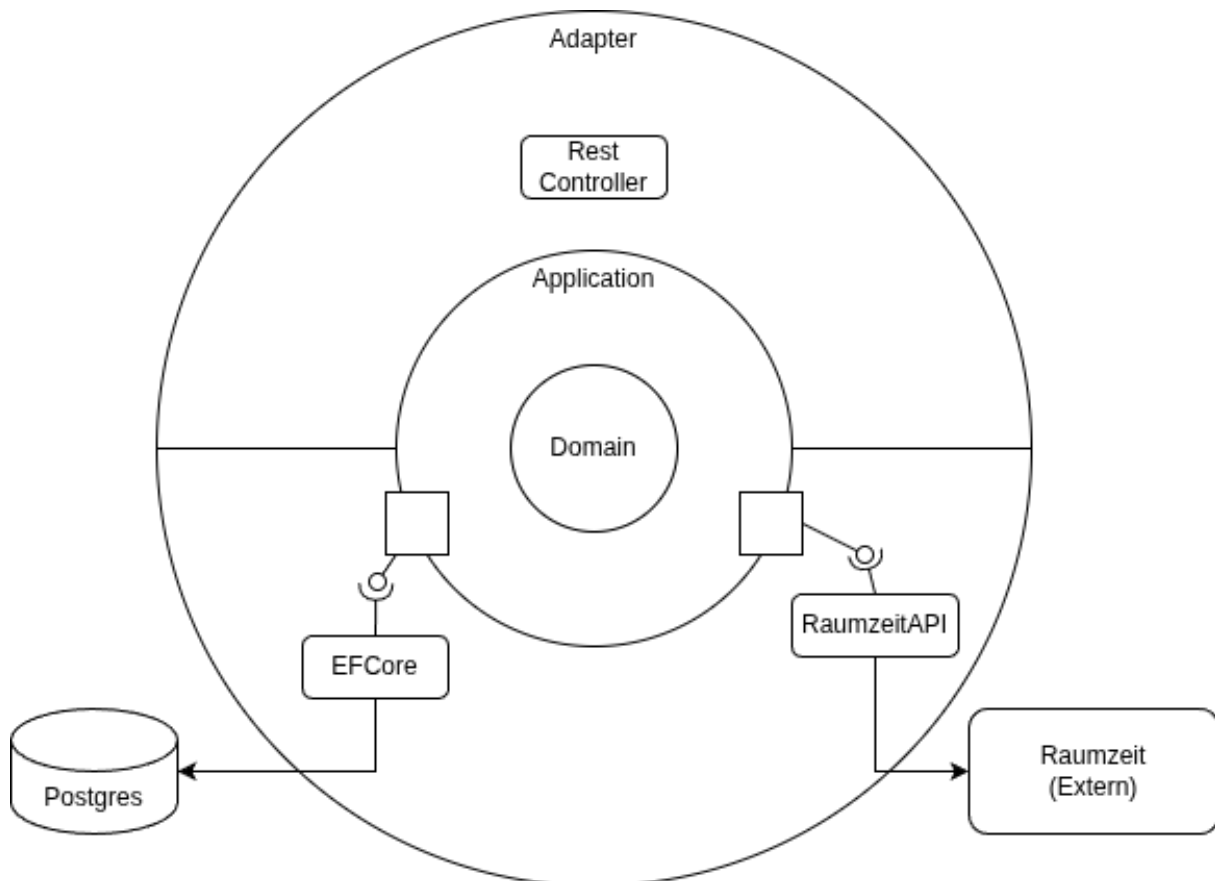


Abbildung 6: Architektur des Projekts

- **Domain:** Hier ist das Domänenmodell in Form von C#-Records definiert. Im C#-Kontext werden Records als Datenhüllen verwendet, da diese standardmäßig immutable (unveränderlich) sind. Dies hilft dabei, unerwartete Seiteneffekte in der Domäne zu vermeiden. Es wird empfohlen, in diesem Projekt den Prinzipien des Domain-Driven Designs (DDD) zu folgen und beispielsweise nach Möglichkeit Value Objects zu verwenden.
- **Application:** In dieser Schicht erfolgt die Verknüpfung der Schichten. Hier werden die Schnittstellen (Ports) in Form von Interfaces definiert. Zudem werden hier Services implementiert, welche die Verarbeitung von Domänenobjekten und die Logik der Anwendungsfälle (Use Cases) definieren. In unserem Beispiel erfolgt hier die Berechnung der Gruppenkalender.
- **Adapter:**
 - **ASP.NET Core Webprojekt:** Hier wird die REST-Schnittstelle implementiert, über die Anwender*innen mit der Anwendung interagieren. In diesem Adapter werden Domänenobjekte zu DTOs (Data Transfer Objects) gemappt, um sie an das Frontend zu übergeben.
 - **EFCore:** Entity Framework Core dient als ORM für die Interaktion mit der Postgres-Datenbank. Hier werden Repositories implementiert, um Domänenobjekte zu speichern und zu laden. Die Domänenmodelle werden nicht direkt via EFCore persistiert, sondern zunächst auf separate Persistenz-Objekte gemappt, um keine technischen Details (wie Datenbank-Annotationen) in die Domäne propagieren zu lassen.
 - **DataProtection:** Hier wird ein Dienst implementiert, der Tokens sicher verschlüsselt, damit keine unverschlüsselten Tokens in die Datenbank geschrieben werden.

- **RaumzeitAPI:** Dieser Adapter realisiert die Anbindung an die externe Raumzeit-Schnittstelle via REST. Über den DataProtection-Adapter werden die Tokens sicher verwaltet. Geladene iCal-Kalender werden mit einer Library geparsed und in das interne Domänenmodell überführt.

7.2.2. API-Endpunkte

Um die Zuständigkeitsbereiche der Adapter klar zu gliedern, wurden diese in die drei Hauptbereiche Authentication, Groups und User unterteilt. Das Adaptermodell wurde wie folgt in die Endpunkte überführt:

Authentication (Quelle: Google) Für die Authentifizierung der Nutzer fiel die Entscheidung auf einen Drittanbieter: in diesem Fall Google. Der Google-Authentifizierungsprozess bietet Nutzern eine komfortable Möglichkeit, sich mit nur einem Schritt bei Apps oder Websites anzumelden, indem sie ihren bestehenden Google-Account verwenden. Damit entfällt die Notwendigkeit separater Zugangsdaten, komplexer Registrierungsformulare und vergessener Passwörter. Mit nur wenigen Codezeilen lässt sich der Login-Prozess integrieren. Weitere Details zum Authentifizierungsprozess finden sich unter (Hyperlink) Kapitel 2.5.

- GET `api/auth/login`: Der Endpunkt leitet den Nutzer mit der entsprechend aufgerufenen URL zur Google-Anmeldeseite weiter.
- GET `api/auth/logout`: Der Nutzer wird abgemeldet.
- GET `api/auth/me`: Der Nutzer registriert sich über den Google-Authentifizierungsprozess in der LernZeit-Anwendung. Entsprechend wird der Nutzer anschließend in der Datenbank hinterlegt.

Group

- GET `api/group/`: Die Response gibt alle vorhandenen Lerngruppen zurück.
- GET `api/group/{groupId}`: Die Response gibt die Gruppe mit der entsprechenden ID zurück.
- POST (protected) `api/group`: Nimmt aus dem Request-Body ein GroupDTO-Objekt sowie den Token des Nutzers entgegen und erstellt daraufhin eine neue Gruppe.
- POST (protected) `api/group/join/{groupId}`: Fügt den eingeloggten Nutzer der Gruppe mit der entsprechenden groupId hinzu.
- POST `api/group/leave/{groupId}`: Entfernt den Nutzer aus der Gruppe mit der entsprechenden groupId.
- GET `api/group/{groupId}/calendar`: Response mit dem Gruppenkalender.

User

- GET `api/user`: Response mit allen registrierten Nutzern.
- GET `api/user/{userId}`: Response des Nutzers mit der entsprechenden ID.
- PUT `api/user`: Nimmt aus dem Request-Body das UserDTO-Objekt des aktualisierten Nutzers entgegen.
- DELETE `api/user/{userId}`: Löscht den Nutzer aus der Datenbank.
- POST `api/user/raumzeit/login`: Nimmt aus dem Request-Body die RaumZeit-Credentials entgegen und ordnet die RaumZeit-Kalenderdaten dem Nutzerkonto zu.
- GET (protected) `api/user/calendar`: Die Response enthält die Kalenderdaten des Nutzers.

7.2.3. Kalenderdaten-Verarbeitung

Die Kernlogik zur Ermittlung gemeinsamer Termine ist im `GroupCalendarService` implementiert. Dieser führte folgende Schritte nacheinander aus:

1. Laden der Gruppe

- Die Gruppe wird anhand ihrer eindeutigen ID aus dem Repository geladen.
- Existiert die Gruppe nicht, wird kein Gruppenkalender erzeugt.

2. Abrufen der persönlichen Kalender

- Für jedes Gruppenmitglied wird der persönliche Kalender über den Kalenderservice geladen.
- Die geladenen Kalender werden zu einer gemeinsamen Sammlung zusammengeführt.

3. Zusammenführen aller Termine

- Alle Termine aus den persönlichen Kalendern werden in einer Liste gesammelt.
- Die Termine werden nach ihrer Startzeit sortiert.

4. Ermittlung der belegten Zeiträume

- Die sortierte Terminliste wird nacheinander durchlaufen.
- Für jeden Termin wird geprüft, ob er sich mit dem zuletzt gespeicherten Zeitraum überschneidet.
 - Keine Überschneidung: Es wird ein neuer Belegungsblock angelegt.
 - Überschneidung: Die Zeiträume werden zu einem gemeinsamen Block zusammengeführt, indem das spätere Enddatum übernommen wird.
- Dadurch entstehen zusammenhängende Zeitintervalle, die alle belegten Zeiten der Gruppenmitglieder abdecken.

7.2.4. Gruppenverwaltung

Gruppen können von jedem Nutzer erstellt werden. Dies geschieht auf der Hauptseite über den Button „Gruppe erstellen„. Im daraufhin erscheinenden Dialogfenster kann ein Gruppenname vergeben werden, nach dessen Bestätigung die Gruppe erstellt wird. Um weitere Nutzer zu einer Gruppe hinzuzufügen, müssen diese vom Ersteller eingeladen werden. Die Einladung kann über einen Link oder das Scannen eines QR-Codes erfolgen, wofür der Nutzer auf der GroupCard auf das QR-Icon tippt.

Der Gruppenkalender kann durch einen Klick auf die Gruppenkarte aufgerufen werden. Er stellt das Ergebnis aller einzelnen Kalender der Gruppenmitglieder dar. Im Gruppenkalender werden ausschließlich bereits belegte Zeitblöcke angezeigt. Durch einen Klick auf eine freie Spalte lässt sich ein Gruppen-Event hinzufügen.

Zum Verlassen einer Gruppe wird der Button „Gruppe verlassen„ oberhalb des Gruppenkalenders betätigt.

Die Verknüpfung zwischen Gruppen und ihren Mitgliedern ist in der Datenbank hinterlegt. Näheres dazu findet sich im folgenden Kapitel. Sind einer Gruppe keine Nutzer mehr zugeordnet, wird diese automatisch aus der Datenbank entfernt.

7.2.5. Datenbank

7.2.5.1. Datenbankstruktur

[Bild Datenbankdiagramm]

Abbildung [Abb. 1] zeigt das Datenbankschema zwischen der User- und der Gruppentabelle. Die User_Groups-Tabelle ist eine Relationstabelle und steht zu den Tabellen users und groups jeweils in einer 1-zu-n-Beziehung. Die Relation wird über die Primärschlüssel von users und groups hergestellt.

7.2.5.2. Implementierung

Das gewählte Datenbankmanagementsystem ist PostgreSQL. Im Rahmen der Domain-Driven-Architektur erfolgt der Datenbankzugriff der LernZeit-Anwendung über den LernZeit.PostgresAdapter. Dieser enthält unter anderem den Datenbankkontext, die für das .NET-Framework EntityFrameworkCore erforderlichen Migrationen sowie die Repositories zu den Gruppen- und Nutzerobjekten, Datenbank-Entity-Klassen und Mapper-Klassen.

Im LernZeitDBContext werden die Datenbank-Entities festgelegt und die UserGroups-Relation hergestellt. Das Group- und das UserRepository stellen diverse Schnittstellen zur Modifizierung der Datenbankobjekte bereit.

7.2.6. Authentifizierung (Google Auth)

Die Sicherheit der Anwendung wird durch die Integration von Google OAuth 2.0 gewährleistet. Das Backend fungiert hierbei als Validierungsschicht:

- Der Client sendet ein von Google ausgestelltes ID-Token.
- Das Backend validiert dieses Token gegen die Google-Server (Signatur- und Audience-Check).
- Nach erfolgreicher Validierung wird die GoogleUserId extrahiert. Diese dient als Primärschlüssel in unserer internen User-Entität, wodurch wir keine sensiblen Passwortdaten speichern müssen.

7.2.7. Kommunikation mit der RaumZeit API

Um den manuellen Aufwand für Studierende zu minimieren, integriert **Lernzeit** die RaumZeit-API der Hochschule.

- **RaumzeitService:** Diese Komponente kapselt die HTTP-Kommunikation mit dem Hochschul-System. Sie übernimmt das Mapping der proprietären RaumZeit-Strukturen auf unsere internen Domain-Modelle.
- **Sicherheit & Token-Management:** Da der Zugriff auf Stundenpläne autorisiert erfolgen muss, speichern wir die notwendigen Zugriffstoken verschlüsselt in der PostgreSQL-Datenbank. Hierfür kommt der TokenEncryptionService zum Einsatz, der die IDataProtectionProvider-Infrastruktur von .NET nutzt.

TODO: Klassendiagramm des Raumzeit-Integrationsmoduls einfügen

Abbildung: Struktur des Raumzeit-Services und dessen Datenfluss

7.3. Deployment und Infrastruktur

Das Projekt verfolgt einen modernen „Infrastructure as Code“ Ansatz.

7.3.1. Deployment-Umgebung

Die Applikation ist so konzipiert, dass sie vollständig in Docker-Containern lauffähig ist. Dies garantiert eine identische Laufzeitumgebung zwischen Entwicklung (Localhost), Testing und

Produktion. Als Datenbank wird eine PostgreSQL-Instanz verwendet, die ebenfalls containerisiert bereitgestellt wird.

7.3.2. Containerisierung und Orchestrierung mit .NET Aspire

Ein technologisches Highlight ist der Einsatz von .NET Aspire. Dies ermöglicht eine nahtlose Orchestrierung der Microservices (oder modularen Monolithen) während der Entwicklung:

- **Service Discovery:** Das Frontend findet das Backend automatisch über logische Namen, statt über statische IP-Adressen.
- **Observability:** .NET Aspire bietet ein integriertes Dashboard, in dem Logs, Traces und Metriken aller Komponenten in Echtzeit eingesehen werden können.
- **Konfigurationsmanagement:** Connection Strings für die Datenbank und API-Keys werden zentral im AppHost verwaltet und sicher an die jeweiligen Services durchgereicht.

TODO: Screenshot des .NET Aspire Dashboards einfügen

Abbildung: Laufzeit-Überwachung der Container-Infrastruktur

8. Fazit

8.1. Rückblick

Im Rahmen dieses Moduls lag der Fokus auf der Software-Architektur des Projekts. Ein besonderes Augenmerk galt dabei der Strukturierung und Einordnung von Zuständigkeitsbereichen. Die Entscheidung fiel auf ein Domain-Driven-Modell, das eine klare Trennung zwischen Anwendungslogik und Adaptern gewährleistet.

Eine zentrale Fragestellung war, welche Nutzerdaten gespeichert werden müssen, damit die Anwendung funktioniert. Das Ziel war es, so wenig wie möglich zu speichern, da der Großteil der nutzerbezogenen Daten von Drittanbietern wie Google Authentication und der RaumZeit-API bereitgestellt wird.

Ebenfalls war die Authentifizierung der Nutzer zu Beginn ein wichtiges Thema. Die Auslagerung dieses Problems an einen Drittanbieter stellte sich als komfortable und zuverlässige Lösung heraus.

Im Frontend stellte die Auswahl einer geeigneten Kalenderkomponente eine Herausforderung dar. Die Entscheidung für eine vorgefertigte React-Kalenderkomponente ermöglichte ein schnelleres Ergebnis, anstatt alle Funktionen von Grund auf selbst zu entwickeln. Da die Auswahl an verfügbaren Komponenten groß ist, wurden mithilfe des KI-Modells OpenCode mehrere Komponenten iterativ getestet, um die für den Anwendungsfall am besten geeignete zu ermitteln.

Im Verlauf des Entwicklungsprozesses kam es vereinzelt dazu, dass Komponenten fehlerhafte Zustände annahmen oder nicht mehr korrekt funktionierten, was eine weitere Herausforderung darstellte.

Darüber hinaus funktionierte die Authentifizierung aus verschiedenen Gründen nicht immer zuverlässig. Insbesondere die CORS-Policies stellten zu Beginn aufgrund von Client-Weiterleitungen ein Problem dar.

Gegen Ende des Projekts ergaben sich in der Testphase weitere Herausforderungen. Das Testen der Anwendung mit mehreren Nutzern ist nicht trivial, da Nutzerkonten eng mit dem jeweiligen Google-Account verknüpft sind. Daher waren zusätzliche Google-Accounts erforderlich, um die Anwendung vollständig testen zu können.

8.2. Ausblick

Das LernZeit-Projekt hat alle Mindestanforderungen erfüllt, die zu Beginn im Pflichtenheft festgelegt wurden. Es ist möglich, Lerngruppen zu erstellen, eigene Kalender aus RaumZeit zu importieren, diese mit den Mitgliedern der Lerngruppe abzugleichen und in einer übersichtlichen Benutzeroberfläche einen gemeinsamen freien Zeitraum festzulegen.

Neben den funktionalen Anforderungen sind auch die nicht-funktionalen Anforderungen erfüllt. Die LernZeit-Anwendung entspricht den Prinzipien des Domain-Driven Designs: Komponenten sind klar in ihren Zuständigkeitsbereichen getrennt, was eine reibungslose Wartung ermöglicht. Ebenso können Komponenten bei Bedarf unkompliziert ausgetauscht oder erweitert werden.

Für zukünftige Erweiterungen wären weitere Use-Cases denkbar. So wäre eine Funktion zur gemeinsamen Terminabstimmung innerhalb einer Gruppe eine sinnvolle Ergänzung. Darüber hinaus würde eine Anzeige aller freien Räume auf dem Campus die Suche nach einem geeigneten Lernort erleichtern. Die Integration von Benachrichtigungen bei neuen Terminvorschlägen oder Änderungen wäre eine nützliche Erinnerungsfunktion. Schließlich wäre auch die Anbindung weiterer Kalender-Apps, beispielsweise Google Calendar oder Outlook, eine denkbare Erweiterung.

9. Literaturverzeichnis

Bibliografie

- [1] Zugriffen: 15. Juni 2025. [Online]. Verfügbar unter: <https://www.figma.com/resource-library/ai-in-design/>
- [2] Zugriffen: 22. Juni 2025. [Online]. Verfügbar unter: <https://in.pinterest.com/pin/53339576832700058/>
- [3] Zugriffen: 22. Juni 2025. [Online]. Verfügbar unter: <https://se.pinterest.com/pin/20969954512913816/>